

Table of Contents

1. Introduction	2
2. Description générale	5
3. Besoins fonctionnels	11
4. Spécification des structures de données	21
5. Spécifications des interfaces externes	28
6. Les besoins en performance	33
7. Les contraintes de développement	36
8. Références	39
9. Index	41
10. Annexes	47

1. Introduction

Buts et destinataires du document

Ce document constitue le rapport de spécification du projet **RhapsodySVN**, réalisé dans le cadre de l'UV DE50. L'objectif de ce projet est la conception et le développement d'un plugin Java pour le logiciel de modélisation **IBM Rational Rhapsody 9.0**, permettant de créer, d'annoter et d'analyser des diagrammes de type **Stakeholder Value Network (SVN)**.

Le projet est réalisé dans un cadre académique et n'a pas vocation à être distribué ou commercialisé en dehors de l'établissement. Ce document s'adresse :

- aux **étudiants** membres de l'équipe de développement, comme référence de conception et d'implémentation ;
- aux **encadrants** du module DE50, pour la validation des choix techniques et fonctionnels ;
- à toute personne souhaitant **comprendre, utiliser ou faire évoluer** le plugin RhapsodySVN.

Définitions et abréviations

Terme / Abréviation	Définition
SVN	Stakeholder Value Network — méthode d'analyse des parties prenantes et de leurs échanges de valeur
SysML	Systems Modeling Language — langage de modélisation de systèmes basé sur UML
UML	Unified Modeling Language — langage de modélisation objet standard
IBM Rhapsody	Outil de modélisation SysML/UML développé par IBM (Rational)
INCOSE	International Council on Systems Engineering
API	Application Programming Interface
JDK	Java Development Kit
JAR	Java ARchive — format d'emballage d'applications Java
UC	Use Case — cas d'utilisation
DFS	Depth-First Search — algorithme de parcours en profondeur d'un graphe
RPUserPlugin	Interface Java de l'API Rhapsody permettant de déclarer un plugin
IRPApplication	Interface Rhapsody représentant l'application en cours d'exécution

IRPProject	Interface Rhapsody représentant le projet ouvert
IRPProfile	Interface Rhapsody représentant un profil SysML
IRPFlow	Interface Rhapsody représentant un flux (arc orienté)
IRPActor	Interface Rhapsody représentant un acteur
IRPCollection	Interface Rhapsody représentant une collection d'éléments du modèle
Stéréotype	Extension du métamodèle UML/SysML permettant de typer un élément
Tag	Propriété supplémentaire associée à un élément via un stéréotype
Value Loop	Cycle dans le graphe SVN passant par le nœud système central

2. Description générale

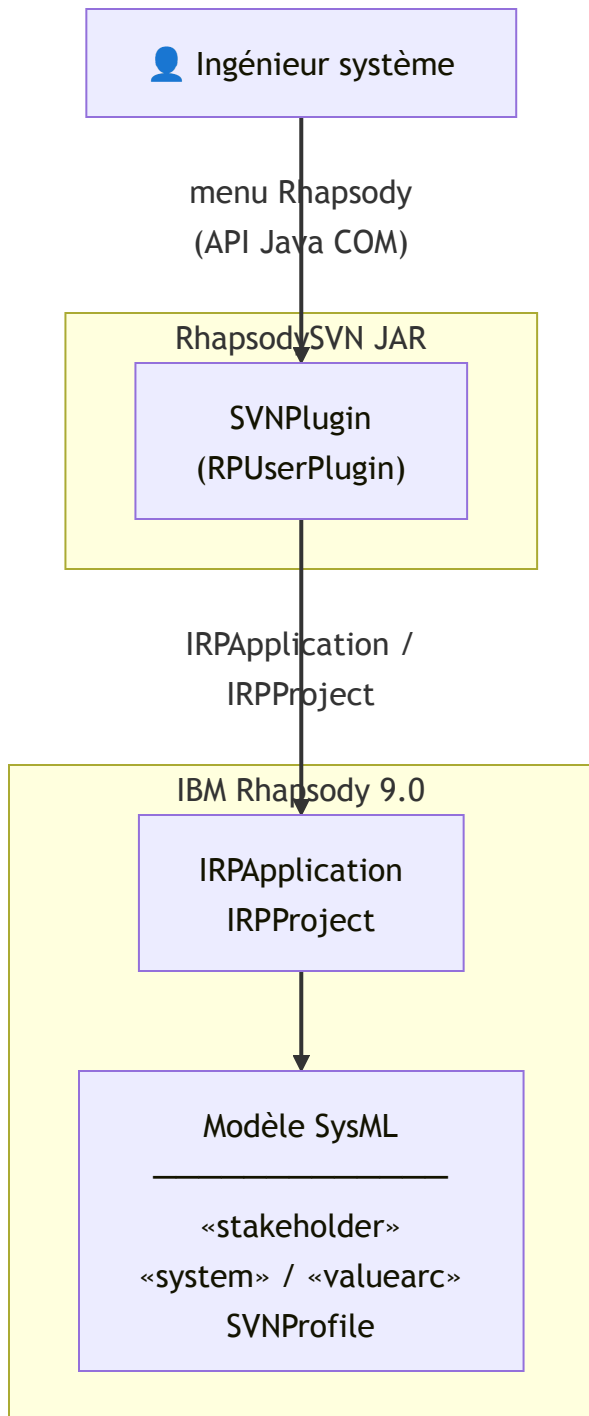
Environnement ou contexte du système

Le plugin **RhapsodySVN** est un composant logiciel qui s'exécute au sein de l'environnement **IBM Rhapsody 9.0**. Il s'intègre à Rhapsody via la classe **SVNPlugin**, qui implémente l'interface **RPUUserPlugin** de l'API Java COM de Rhapsody.

Il interagit avec trois entités principales :

- **IBM Rhapsody**: l'application hôte, avec laquelle le plugin communique via l'API Java COM (**RhapsodyAppServer**). Rhapsody doit être ouvert et un projet actif doit être chargé pour que le plugin fonctionne. Le plugin s'intègre au menu contextuel de Rhapsody via la classe **SVNPlugin**.
- **Le modèle SysML/UML**: le projet Rhapsody ouvert, qui contient les éléments manipulés par le plugin (acteurs «**stakeholder**», classes «**system**», flux «**valuearc**», diagrammes «**SVNDiagram**»). Le plugin lit et modifie ce modèle directement via l'API Rhapsody.
- **L'utilisateur**: déclenche les commandes du plugin depuis le menu de Rhapsody ou en ligne de commande, et crée manuellement les éléments SVN dans le modèle.

Le schéma ci-dessous représente ces interactions :

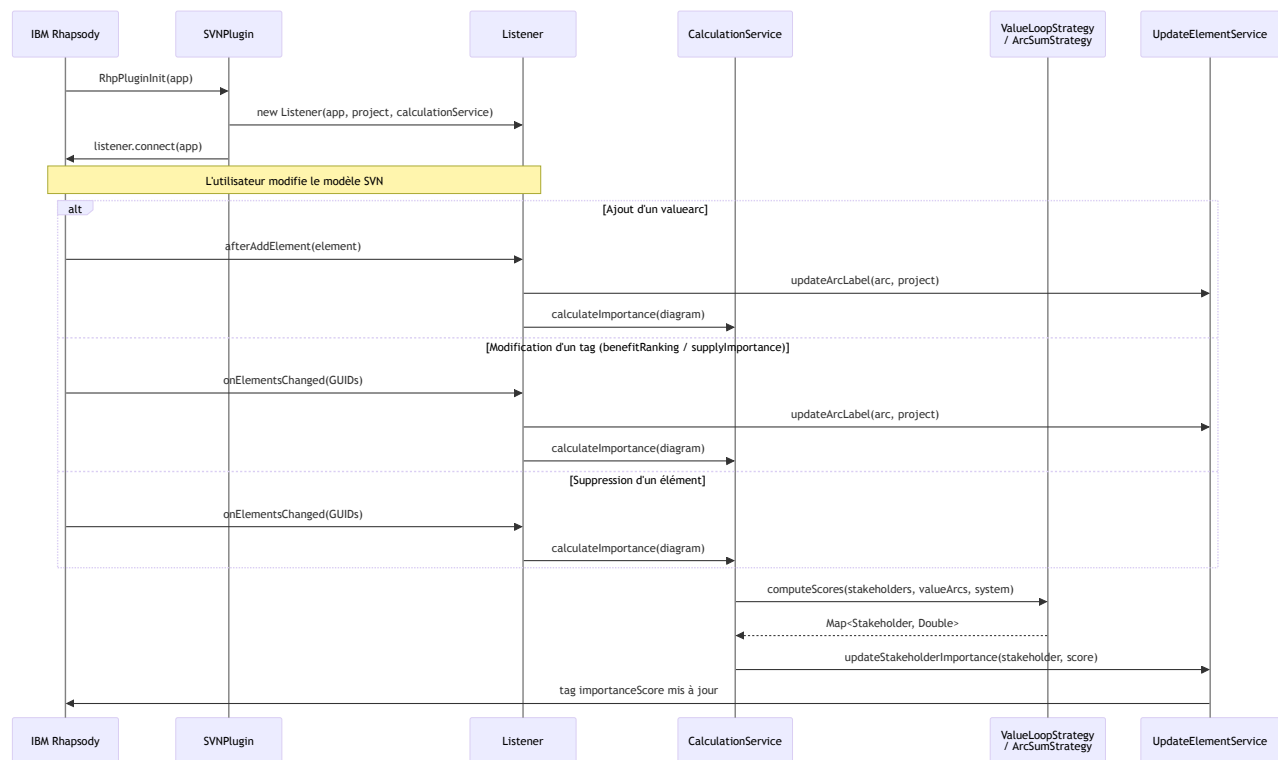


Le plugin n'a pas de persistance propre : toutes les données (scores, stéréotypes, tags) sont stockées directement dans le modèle Rhapsody. Il ne communique avec aucun service réseau ou base de données externe.

Architecture en couches

Le code est organisé selon une architecture en couches :

Couche	Classes	Rôle
Point d'entrée	SVNPlugin, Main (permet de lancer le programme sans Rhapsody)	Initialisation du plugin, connexion à Rhapsody
Contrôleur réactif	Listener	Écoute les événements Rhapsody (afterAddElement, onElementsChanged) et orchestre les mises à jour
Services (logique métier)	CalculationService, UpdateElementService	Calcul d'importance SVN et écriture des résultats dans le modèle
Stratégies de calcul	ValueLoopStrategy, ArcSumStrategy	Algorithmes interchangeables (pattern Strategy)
Modèles	Stakeholder, ValueArc, SVNSystem, ValueLoop	Wrappers des interfaces Rhapsody
Wrapper Rhapsody	RhapsodyWrapper	Accès bas niveau à l'API Rhapsody
Constantes	SVNConstants	Centralisation des noms de stéréotypes et de tags



Caractéristiques des utilisateurs

Ce plugin s'adresse principalement à des **ingénieurs systèmes** (professionnels ou étudiants).

Caractéristique	Description
Domaine	Ingénierie système, modélisation SysML/UML
Outil maîtrisé	IBM Rhapsody (utilisateur régulier)
Connaissance méthode	Familier avec la notion de Stakeholder Value Network (SVN), les équations de Cameron (2007) et les matrices de score INCOSE 2018
Fréquence d'utilisation	Occasionnelle
Compétences informatiques	Intermédiaires à avancées (capable d'utiliser un outil de modélisation professionnel)

📖 L'utilisateur n'a pas besoin de connaître Java ni l'API Rhapsody pour utiliser le plugin : toutes les interactions passent par le menu graphique de Rhapsody ou par des boîtes de dialogue Swing générées par le plugin.

Les contraintes principales de développement

Contraintes techniques

Composant	Contrainte
Langage	Java 8
JDK	JDK 1.8
IBM Rhapsody	Version 9.0 obligatoire (le JAR <code>rhapsody.jar</code> utilisé est spécifique à cette version)
Système d'exploitation	Windows uniquement
Outil de build	Maven 3.x
Dépendance externe	<code>rhapsody.jar</code> fourni par IBM, à placer dans <code>lib/</code>

Normes et standards

- Algorithme de calcul conforme aux **équations SVN de Cameron (2007)** (importance relative des stakeholders)
- Matrice de score des arcs conforme à la **Figure 3 de l'INCOSE 2018**
- Modélisation via des **stéréotypes SysML** définis dans un profil Rhapsody dédié (`SVNProfile`)

Organisation du code

Le projet suit une architecture en couches : commandes (point d'entrée), services (logique métier), wrapper Rhapsody (accès bas niveau à l'API) et constantes. Chaque commande est une classe Java indépendante exécutable en ligne de commande ou via le plugin. Le package Java est `fr.utbm.RhapsodySVN`.

Hypothèses de travail

Hypothèse	Impact si non respectée
IBM Rhapsody 9.0 est ouvert avec un projet actif au moment du lancement	Le plugin ne peut pas se connecter à l'API — toutes les commandes échouent
Un package <code>Default</code> existe dans le projet Rhapsody	Les commandes de création et de nettoyage ne trouvent pas leur cible
La commande <code>SVN Configure</code> a été exécutée avant toute autre	Les stéréotypes et tags nécessaires (« <code>stakeholder</code> », « <code>valuearc</code> », <code>importanceScore</code> , etc.) sont absents du modèle
Des éléments « <code>stakeholder</code> », « <code>system</code> » et « <code>valuearc</code> » ont été créés dans le modèle	<code>SVNCalculateCommand</code> ne trouve aucun nœud à traiter et ne produit aucun résultat
L'arc « <code>valuearc</code> » cible est sélectionné graphiquement dans Rhapsody	<code>SVNEditArcCommand</code> et <code>SVNArcColorCommand</code> ne peuvent pas identifier l'élément à modifier

En cas de modèle sans nœud «`system`», le plugin bascule automatiquement sur un **calcul simplifié par somme des arcs** (fallback implémenté dans `CalculationService`), ce qui constitue la principale solution de repli fonctionnelle.

3. Besoins fonctionnels

Le plugin expose **huit cas d'utilisation**, tous déclenchés par l'ingénieur système depuis le menu contextuel de Rhapsody ou en ligne de commande. Ils sont regroupés en trois grandes fonctionnalités : **initialisation**, **analyse SVN**, et **visualisation**.

Groupe	Cas d'utilisation
Initialisation	UC1 — SVN Configure, UC8 — SVN Clean
Édition du modèle	UC2 — SVN Create Arc, UC3 — SVN Edit Arc
Analyse SVN	UC4 — SVN Calculate
Visualisation	UC5 — SVN Colorize Stakeholders, UC6 — SVN Set Arc Color, UC7 — SVN Update Arc Labels

UC1 — Configurer le profil SVN

Champ	Contenu
Nom	SVN Configure
Description	Crée ou recrée le profil <code>SVNProfile</code> dans le projet Rhapsody actif. Initialise les stéréotypes («stakeholder», «system», «valuearc», «SVNDiagram»), les types énumérés (<code>BenefitRanking</code> , <code>SupplyImportance</code>) et les tags associés.
Acteurs concernés	Ingénieur système
Entrées et provenances	Projet Rhapsody actif (via <code>IRPApplication.activeProject()</code>)
Traitement	<code>ProfileService.configureProfile(true)</code> — supprime l'ancien profil s'il existe, puis recrée tous les éléments du profil.
Sorties	Le profil <code>SVNProfile</code> est présent dans le projet. Message console <code>[SVN] Plugin initialisé</code> . Aucun retour visuel à l'utilisateur.

UC2 — Créer un arc de valeur

Champ	Contenu
Nom	SVN Create Arc
Description	Crée un arc «valuearc» entre deux éléments sélectionnés dans le diagramme Rhapsody.
Acteurs concernés	Ingénieur système
Entrées et provenances	Deux éléments sélectionnés graphiquement dans Rhapsody (source et cible de l'arc)
Traitement	DiagramService.createArcBetweenSelected() — récupère la sélection courante via IRPApplication.getSelectedGraphElements(), vérifie que les deux nœuds portent un stéréotype SVN («stakeholder» ou «system»), crée un IRP Flow avec le stéréotype «valuearc», puis l'insère graphiquement dans le diagramme.
Sorties	Un arc «valuearc» apparaît dans le diagramme entre les deux éléments sélectionnés. Message console confirmant la création.

UC3 — Éditer les pondérations d'un arc

Champ	Contenu
Nom	SVN Edit Arc
Description	Permet à l'utilisateur de renseigner les deux tags de pondération d'un arc «valuearc» sélectionné : benefitRanking et supplyImportance.
Acteurs concernés	Ingénieur système
Entrées et provenances	Arc «valuearc» sélectionné dans Rhapsody ; choix de l'utilisateur via deux boîtes de dialogue Swing (JOptionPane)
Traitement	Vérifie que l'élément sélectionné est bien un IRPFlow portant le stéréotype «valuearc». Ouvre deux dialogues de sélection successifs (listes déroulantes), puis écrit les valeurs choisies dans les tags de l'arc et sauvegarde le projet.
Sorties	Les tags benefitRanking et supplyImportance de l'arc sont mis à jour. Message d'avertissement si l'élément sélectionné n'est pas un «valuearc».

Valeurs possibles :

Tag	Valeurs
benefitRanking	MIGHT_BE, SHOULD_BE, MUST_BE
supplyImportance	LOW, MEDIUM, HIGH

UC4 — Calculer l'importance des stakeholders

Champ	Contenu
Nom	SVN Calculate
Description	Calcule le score d'importance normalisé de chaque «stakeholder» du modèle en appliquant les équations SVN (Cameron 2007 / INCOSE 2018).
Acteurs concernés	Ingénieur système
Entrées et provenances	Projet Rhapsody actif contenant des éléments «stakeholder», «system» et «valuearc» avec leurs tags benefitRanking et supplyImportance renseignés
Traitement	CalculationService.calculateImportance() — parcourt récursivement le modèle, construit le graphe des arcs, recherche les <i>value loops</i> par DFS depuis le nœud «system», puis calcule l'importance relative de chaque stakeholder (Équation 2, Cameron 2007). En l'absence de nœud «system», bascule sur un calcul simplifié par somme des arcs.
Sorties	Le tag importanceScore (valeur entre 0 et 1) de chaque «stakeholder» est mis à jour dans le modèle. Messages console détaillant les loops trouvés et les scores calculés.

Algorithme de calcul (méthode principale) :

1. Construire le graphe orienté des arcs «valuearc»
2. Trouver tous les cycles (value loops) depuis le nœud «system» par DFS

3. **Équation 1:** score d'un loop = produit des scores de ses arcs

4. **Équation 2:** importance(stakeholder S) = Σ scores des loops contenant S / Σ scores de tous les loops

Matrice de score des arcs (INCOSE 2018, Figure 3) :

supplyImportance ↓ \ benefitRanking →	MIGHT_BE	SHOULD_BE	MUST_BE
HIGH	0.30	0.50	0.95
MEDIUM	0.20	0.40	0.80
LOW	0.10	0.20	0.40

UC5 — Coloriser les stakeholders

Champ	Contenu
Nom	SVN Colorize Stakeholders
Description	Applique une couleur de remplissage à chaque nœud «stakeholder» dans les diagrammes SVN, en fonction de leur rang d'importance calculé.
Acteurs concernés	Ingénieur système
Entrées et provenances	Modèle Rhapsody avec des «stakeholder» dont le tag importanceScore est renseigné (nécessite l'exécution préalable de UC4)
Traitement	DiagramService.colorizeStakeholdersByRank() — trie les stakeholders par score décroissant, les divise en trois tiers, applique les couleurs rouge/orange/jaune selon l'importance, et affiche le score dans le nom affiché.
Sorties	Les nœuds «stakeholder» sont colorisés dans les diagrammes SVN (rouge = plus importants, orange = importants, jaune = moins importants).

Code couleur appliqué :

Tier	Couleur	Signification
Tiers supérieur (1/3 le plus important)	Rouge (#FF4444)	Stakeholders critiques
Tiers médian	Orange (#FFA500)	Stakeholders importants
Tiers inférieur	Jaune (#FFFF00)	Stakeholders secondaires

UC6 — Coloriser un arc de valeur

Champ	Contenu
Nom	SVN Set Arc Color
Description	Permet à l'utilisateur de choisir une couleur personnalisée pour un arc «valuearc» sélectionné dans le diagramme.
Acteurs concernés	Ingénieur système
Entrées et provenances	Arc «valuearc» sélectionné dans Rhapsody ; couleur choisie via une boîte de dialogue JColorChooser
Traitement	DiagramService.setArcColor() — vérifie que l'élément sélectionné est un «valuearc», puis applique la couleur choisie (propriété graphique LineColor) sur toutes les représentations graphiques de l'arc dans les diagrammes SVN.
Sorties	La couleur de l'arc est mise à jour dans le ou les diagrammes SVN. Message console confirmant l'application de la couleur.

UC7 — Mettre à jour les labels d'arcs

Champ	Contenu
Nom	SVN Update Arc Labels
Description	Affiche les valeurs de pondération (<code>benefitRanking</code> et <code>supplyImportance</code>) directement sur chaque arc «valuearc» dans les diagrammes SVN.
Acteurs concernés	Ingénieur système
Entrées et provenances	Projet Rhapsody actif contenant des arcs «valuearc» avec leurs tags renseignés
Traitement	<code>DiagramService.updateArcLabels()</code> — parcourt tous les arcs «valuearc» du modèle et met à jour leur <code>displayName</code> avec le format <code>B:<benefitRanking> S:<supplyImportance></code> . En cas d'échec de <code>setDisplayName</code> , tente une mise à jour via les propriétés graphiques des éléments de diagramme.
Sorties	Les arcs affichent leurs pondérations dans le diagramme (ex. : <code>B:MUST_BE S:HIGH</code>).

UC8 — Nettoyer le modèle

Champ	Contenu
Nom	SVN Clean
Description	Supprime tous les éléments SVN du package Default du projet Rhapsody (stakeholders, systems, valuearcs, diagrammes SVN) ainsi que le profil SVNProfile. Remet le modèle dans un état vierge.
Acteurs concernés	Ingénieur système
Entrées et provenances	Projet Rhapsody actif
Traitement	ProfileService.cleanDefaultPackage() puis ProfileService.deleteProfile() — collecte tous les éléments SVN avant suppression, supprime d'abord les arcs, puis les nœuds, puis les diagrammes, dans le bon ordre pour éviter les erreurs de dépendance.
Sorties	Le modèle ne contient plus d'éléments SVN. Le profil SVNProfile est supprimé. Message console indiquant le nombre d'éléments supprimés.

4. Spécification des structures de données

Le plugin RhapsodySVN ne possède pas de base de données propre. Toutes les données sont stockées directement dans le **modèle Rhapsody** sous forme de stéréotypes, de tags et de types énumérés. Cette section décrit les entités manipulées, leurs relations et leurs attributs.

Modèle d'entités

```
SVNProfile
├─ Stéréotype «stakeholder» (métaclasse Actor)
│   └─ tag: importanceScore (Float)
├─ Stéréotype «system» (métaclasse Class)
├─ Stéréotype «valuearc» (métaclasse Flow)
│   └─ tag: benefitRanking (BenefitRanking)
│       └─ tag: supplyImportance (SupplyImportance)
├─ Stéréotype «SVNDiagram» (métaclasse ObjectModelDiagram)
├─ Type BenefitRanking (Enumeration)
│   └─ MIGHT_BE
│   └─ SHOULD_BE
│   └─ MUST_BE
└─ Type SupplyImportance (Enumeration)
    └─ LOW
    └─ MEDIUM
    └─ HIGH
```

Entité : «stakeholder»

Propriété	Valeur
Nom	stakeholder
But	Représente une partie prenante du système (acteur humain ou organisation) dont on souhaite évaluer l'importance relative dans le réseau de valeur
Métaclasse Rhapsody	Actor (IRPActor)
Profil	SVNProfile
Tag	importanceScore (Float, entre 0 et 1) — score d'importance calculé par UC4
Relations	Peut être relié à d'autres «stakeholder» ou à un «system» via des arcs «valuearc»

Entité : «system»

Propriété	Valeur
Nom	system
But	Représente le système central du réseau SVN. C'est le nœud de départ et d'arrivée des <i>value loops</i> utilisées dans le calcul de Cameron (2007). Un seul nœud «system» est attendu par modèle.
Métaclasses Rhapsody	Class (IRPCClass)
Profil	SVNProfile
Tags	Aucun tag spécifique
Relations	Connecté aux «stakeholder» via des «valuearc»

Entité : «valuearc»

Propriété	Valeur
Nom	valuearc
But	Représente un flux de valeur orienté entre deux entités du réseau SVN (stakeholder ↔ system ou stakeholder ↔ stakeholder). Chaque arc est pondéré par deux critères.
Métaclasses Rhapsody	Flow (IRPFlow)
Profil	SVNProfile
Tag benefitRanking	Type BenefitRanking — importance du bénéfice apporté par cet arc du point de vue du destinataire (MIGHT_BE, SHOULD_BE, MUST_BE)
Tag supplyImportance	Type SupplyImportance — capacité du fournisseur à satisfaire ce flux de valeur (LOW, MEDIUM, HIGH)
Relations	Orienté entre deux entités SVN : end1 (source) et end2 (cible)

Entité : «SVNDiagram»

Propriété	Valeur
Nom	SVNDiagram
But	Diagramme Rhapsody dédié à la représentation visuelle d'un réseau SVN. Il regroupe les nœuds «stakeholder» et «system» et les arcs «value arc».
Métaclass e Rhapsod y	ObjectModelDiagram (IRPObjectModelDiagram)
Profil	SVNProfile
Propriété	DrawingToolbar configurée pour exposer les outils stakeholder, system et valuearc
Relations	Contient les représentations graphiques (IRPGraphElement) des entités SVN

Type énuméré : BenefitRanking

Propriété	Valeur
Nom	BenefitRanking
But	Qualifie l'importance du bénéfice que représente un arc «valuearc» pour son destinataire
Littérature	MIGHT_BE, SHOULD_BE, MUST_BE (ordre croissant d'importance)
Utilisation	Tag benefitRanking du stéréotype «valuearc»

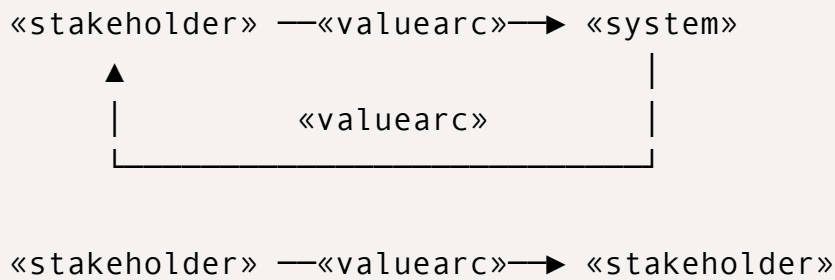
Type énuméré : SupplyImportance

Propriété	Valeur
Nom	SupplyImportance
But	Qualifie la capacité du fournisseur à satisfaire le flux de valeur représenté par un arc
Littéraux	LOW, MEDIUM, HIGH (ordre croissant de capacité)
Utilisation	Tag supplyImportance du stéréotype «valuearc»

Profil SVNProfile

Propriété	Valeur
Nom	SVNProfile
But	Profil SysML/UML créé dans le projet Rhapsody actif, qui regroupe tous les stéréotypes et types nécessaires au fonctionnement du plugin
Contenu	Stéréotypes «stakeholder», «system», «valuearc», «SVNDiagram»; types BenefitRanking, SupplyImportance
Gestion	Créé ou recréé par UC1. Supprimé par UC8. Stocké dans le modèle Rhapsody (pas de fichier externe).

Relations entre entités



Un réseau SVN typique comprend :

- Un nœud «system» central
- Plusieurs nœuds «stakeholder» autour
- Des arcs «valuearc» orientés formant des cycles (value loops) passant par le système

5. Spécifications des interfaces externes

Interface matériel / logiciel

Configuration minimale

Composant	Configuration minimale
Système d'exploitation	Windows (7, 10 ou 11) — Windows uniquement
Processeur	Compatible avec l'exécution d'IBM Rhapsody 9.0
Mémoire RAM	4 Go minimum (8 Go recommandés, requis par IBM Rhapsody)
Espace disque	500 Mo (installation Rhapsody non comprise)
JDK	JDK 1.8 (Java 8) installé et accessible dans le PATH

Périphériques requis

Périphérique	Usage
Écran	Affichage des diagrammes Rhapsody et des boîtes de dialogue Swing
Souris	Sélection des éléments graphiques dans les diagrammes (UC2, UC3, UC6)
Clavier	Saisie de noms d'éléments dans Rhapsody

Interface logiciel / logiciel

IBM Rhapsody 9.0

Propriété	Valeur
Nom	IBM Rational Rhapsody
Version	9.0 (obligatoire — l'API COM utilisée est spécifique à cette version)
Provenance	Fourni par IBM. Disponible via les licences académiques UTBM.
But d'utilisation	Application hôte du plugin. Fournit l'API Java via <code>rhapsody.jar</code> .
Interface	<code>IRPApplication</code> , <code>IRPPProject</code> , <code>IRPPProfile</code> , <code>IRPFlow</code> , <code>IRPActor</code> , <code>IRPCollection</code> , <code>RPUUserPlugin</code> , <code>RhapsodyAppServer</code>
Mode d'accès	Appel de <code>RhapsodyAppServer.getActiveRhapsodyApplication()</code> pour obtenir l'application active

Le JAR `rhapsody.jar` est inclus comme dépendance locale dans `lib/rhapsody.jar` et déclaré dans `pom.xml` avec `<scope>system</scope>`.

Maven 3.x

Propriété	Valeur
Nom	Apache Maven
Version	3.x
But	Outil de build et de gestion des dépendances
Usage	Compilation du projet, packaging en JAR (mvn package)

Java Swing (javax.swing)

Propriété	Valeur
Nom	Java Swing
Version	Incluse dans JDK 1.8
But	Affichage des boîtes de dialogue interactives du plugin
Classes utilisées	JOptionPane (sélection des pondérations UC3), JColorChooser (sélection de couleur UC6)

Interface Homme / logiciel

Menu contextuel Rhapsody

Le plugin s'intègre dans le menu de Rhapsody via la classe SVNPlugin. Les commandes disponibles depuis le menu sont :

Entrée de menu	Cas d'utilisation
SVN Configure	UC1 — Configurer le profil
SVN Create Arc	UC2 — Créer un arc de valeur
(via sélection + menu)	UC3 — Éditer les pondérations
SVN Calculate	UC4 — Calculer l'importance
SVN Colorize Stakeholders	UC5 — Coloriser les stakeholders
SVN Set Arc Color	UC6 — Coloriser un arc
SVN Update Arc Labels	UC7 — Mettre à jour les labels
SVN Clean	UC8 — Nettoyer le modèle

Boîtes de dialogue Swing

Deux commandes ouvrent des fenêtres interactives :

UC3 — SVN Edit Arc: deux boîtes `JOptionPane.showInputDialog` successives permettent de sélectionner :

1. La valeur de `benefitRanking` parmi `{MIGHT_BE, SHOULD_BE, MUST_BE}`
2. La valeur de `supplyImportance` parmi `{LOW, MEDIUM, HIGH}`

UC6 — SVN Set Arc Color: une fenêtre `JColorChooser` permet à l'utilisateur de choisir librement une couleur pour l'arc sélectionné.

Messages console

Le plugin communique son état interne via des messages sur la sortie standard (console Rhapsody ou terminal) :

Préfixe	Signification
[SVN]	Message d'information (opération normale)
[SVN] (sur System.err)	Message d'erreur ou d'avertissement

Exemples de messages produits :

- [SVN] Plugin initialisé. — à l'ouverture du plugin
- [SVN] Importance Stakeholder1 = 0.3750 — après UC4
- [SVN] Nettoyage terminé : 7 élément(s) supprimé(s). — après UC8
- [SVN] Commande inconnue : <nom> — si une entrée de menu non reconnue est déclenchée

Spécification des messages d'erreur

Situation	Message affiché
Élément sélectionné non valide pour UC3	Boîte JOptionPane WARNING : <i>"Sélectionnez un arc «valuearc» dans le diagramme."</i>
Élément sélectionné non valide pour UC3	Boîte JOptionPane WARNING : <i>"L'élément sélectionné n'est pas un «valuearc»."</i>
Moins de 2 éléments sélectionnés pour UC2	Message console : <i>"Sélectionnez 2 éléments (stakeholder ou system) dans le diagramme."</i>
Aucun projet actif	Message console : <i>"Aucun projet actif."</i>
Aucun stakeholder trouvé pour UC4	Message console : <i>"Aucun stakeholder trouvé."</i>

6. Les besoins en performance

Temps de réponse

Le plugin est conçu pour une utilisation interactive au sein d'IBM Rhapsody. Les temps de réponse acceptables sont définis par rapport à la perception utilisateur :

Opération	Temps de réponse attendu	Commentaire
UC1 — SVN Configurer	< 5 secondes	Création du profil, des stéréotypes et des types dans Rhapsody
UC2 — SVN Create Arc	< 2 secondes	Création d'un flow et insertion graphique dans le diagramme
UC3 — SVN Edit Arc	< 1 seconde (hors saisie utilisateur)	Lecture/écriture de tags uniquement
UC4 — SVN Calculer	< 10 secondes pour ≤ 50 nœuds	Dépend de la complexité du graphe (recherche DFS de cycles)
UC5 — SVN Colorize Stakeholders	< 5 secondes	Parcours des éléments graphiques des diagrammes
UC6 — SVN Set Arc Color	< 2 secondes (hors saisie utilisateur)	Modification d'une propriété graphique
UC7 — SVN Update Arc Labels	< 5 secondes	Parcours de tous les arcs et mise à jour des labels
UC8 — SVN Clean	< 10 secondes	Suppression de tous les éléments SVN du modèle

Taille du modèle

Paramètre	Valeur maximale recommandée
Nombre de nœuds «stakeholder»	50
Nombre d'arcs «valuearc»	200
Nombre de diagrammes «SVNDiagram»	10
Profondeur maximale des value loops	Non limitée, mais performance dégradée au-delà de 20 nœuds

Complexité algorithmique

L'algorithme de recherche de value loops (UC4) est basé sur un **DFS itératif** depuis le nœud «system». Sa complexité est :

- **Cas favorable:** $O(N + A)$ avec N nœuds et A arcs — graphe sans cycles ou avec peu de cycles
- **Cas défavorable:** $O(N! / (N-k)!)$ avec k = longueur maximale d'un cycle — graphe fortement connexe

Pour les cas d'usage académiques typiques (< 20 stakeholders), les performances sont acceptables sans optimisation particulière. Pour des modèles plus grands, le fallback par somme des arcs reste disponible.

Contraintes liées à l'environnement

- Le plugin s'exécute **dans le même processus JVM** qu'IBM Rhapsody. Il ne peut pas monopoliser le thread principal sous peine de bloquer l'interface Rhapsody.

- Les opérations de longue durée (UC4 sur grand modèle) s'exécutent de manière **synchrone** dans la version actuelle — aucun thread séparé n'est utilisé. L'interface Rhapsody sera temporairement non réactive pendant le calcul.
- Le plugin ne gère pas de **transactions simultanées**: une seule commande peut être en cours d'exécution à la fois.

7. Les contraintes de développement

Fiabilité et tolérance aux fautes

Le plugin doit fonctionner de manière robuste même en cas de modèle Rhapsody incomplet ou mal configuré. Les principales mesures de fiabilité implémentées sont :

Situation anormale	Comportement du plugin
Aucun projet Rhapsody actif	Log d'erreur console, arrêt gracieux de la commande sans exception non gérée
Stéréotype ou tag absent du modèle	Vérification préalable via <code>RhapsodyWrapper.hasStereotype()</code> avant toute manipulation
Élément sélectionné non conforme	Message d'avertissement à l'utilisateur (boîte <code>JOptionPane</code> ou log console)
Absence de nœud «system»	Fallback automatique sur le calcul simplifié par somme des arcs
Échec d'une opération API Rhapsody	Les exceptions sont catchées localement avec un log d'erreur ; l'opération est ignorée sans arrêt du plugin
Suppression d'un élément déjà supprimé	Tentative de <code>deleteFromProject()</code> en fallback si la méthode spécifique échoue

Le plugin **ne lève jamais d'exception non gérée** susceptible de planter IBM Rhapsody. Toutes les opérations à risque sont encadrées par des blocs `try/catch`.

Comportement dans les situations anormales

Les exceptions critiques sont traitées selon la politique suivante :

- **Exceptions lors de la création du profil:** log d'erreur, abandon de l'opération en cours, le profil peut être dans un état partiel
- **Exceptions lors du calcul:** log d'erreur, le score concerné est ignoré ou remis à 0
- **Exceptions lors des opérations graphiques:** log d'erreur, tentative d'une méthode alternative (ex. : `setGraphicalPropertyOfText` si `setDisplayNames` échoue)

Sécurité

Le plugin ne gère pas de données sensibles (mots de passe, informations personnelles, données réseau). Les contraintes de sécurité se limitent à :

- **Restriction d'accès:** le plugin n'expose pas d'API externe. Il ne peut être déclenché que depuis IBM Rhapsody (menu ou ligne de commande locale).
- **Contrôle des données:** les valeurs de tags (`benefitRanking`, `supplyImportance`) sont validées contre des listes de valeurs prédéfinies. Tout tag absent ou invalide est traité comme une valeur par défaut (`MIGHT_BE`/ score 0.2).
- **Pas de persistance externe:** aucune donnée n'est écrite en dehors du modèle Rhapsody. Pas de fichier temporaire, pas de connexion réseau.

Utilisation de standards

Domaine	Standard utilisé
Méthode de calcul SVN	Équations de Cameron (2007) — voir Annexe A
Matrice de score des arcs	Figure 3, INCOSE 2018 — voir Annexe B
Langage de modélisation	SysML (via profil Rhapsody)
Langage de programmation	Java 8 (conformité JDK 1.8)
Outil de build	Maven 3.x (convention de structure <code>src/main/java</code>)
Encodage	UTF-8 (déclaré dans <code>pom.xml</code>)

Maintenabilité

Le code est structuré pour faciliter son évolution :

- **Séparation des responsabilités:** les commandes ne contiennent pas de logique métier ; celle-ci est déléguée aux services.
- **Centralisation des constantes:** tous les noms de stéréotypes, tags et types sont définis dans `SVNConstants`, évitant les magic strings dispersées dans le code.
- **Wrapper d'API:** `RhapsodyWrapper` isole les appels à l'API Rhapsody, facilitant les adaptations à d'autres versions.

Contraintes de déploiement

Le plugin est distribué sous forme d'un **JAR autonome** (`RhapsodySVN-1.0-SNAPSHOT.jar`) à placer dans le répertoire d'installation de Rhapsody. La configuration du menu Rhapsody doit être effectuée manuellement dans le fichier de configuration de Rhapsody pour déclarer le plugin et ses entrées de menu.

8. Références

Références bibliographiques

Référence	Description
Cameron (2007)	B. Cameron, <i>"Stakeholder Value Network Analysis"</i> , INCOSE 2007. Définit la méthode SVN et les équations de calcul d'importance relative des parties prenantes (Équations 1 et 2).
INCOSE 2018	International Council on Systems Engineering, <i>Systems Engineering Handbook</i> , version 4, 2018. La Figure 3 définit la matrice de score des arcs utilisée pour calculer le poids de chaque «valuearc».
IEEE Std 830-1998	IEEE, <i>Recommended Practice for Software Requirements Specifications</i> , 1998. Norme de référence pour la structure du présent document de spécification.
Sommerville (1989)	I. Sommerville, <i>"Software Engineering"</i> , 3rd edn. Wokingham : Addison-Wesley, 1989.
Satzinger et al. (2002)	J. Satzinger, R.B. Jackson, S. Burd, <i>"Analyse et conception des systèmes d'information"</i> , Editions Reynold Goulet, 2002.
Jacobson et al. (1999)	I. Jacobson, G. Booch, J. Rumbaugh, <i>"The Unified Software Development Process"</i> , Addison-Wesley, 1999.
Koukam (2006)	A. Koukam, <i>"Software Engineering"</i> , Course "Introduction to Software Engineering", University of Shanghai, China, 2006.

Documentation technique

Référence	Description
IBM Rhapsody API	Documentation de l'API Java COM d'IBM Rational Rhapsody 9.0. Interfaces <code>IRPApplication</code> , <code>IRPPProject</code> , <code>IRPPProfile</code> , <code>IRPFlow</code> , <code>IRPActor</code> , <code>RPUserPlugin</code> , etc.
Java SE 8 API	Documentation officielle Oracle du JDK 1.8, incluant <code>javax.swing</code> (<code>JOptionPane</code> , <code>JColorChooser</code>).
Apache Maven	Documentation officielle Maven 3.x. Structure de projet <code>src/main/java</code> , gestion des dépendances <code>pom.xml</code> .

9. Index

Index alphabétique des termes

Terme	Section(s)
API	§1, §2, §5
Arc de valeur («valuearc»)	§2, §3 (UC2, UC3, UC6, UC7), §4, §5
BenefitRanking	§3 (UC3, UC4), §4, Annexe B
CalculationService	§2, §3 (UC4)
Cameron (2007)	§3 (UC4), §7, §8, Annexe A
DiagramService	§2, §3 (UC2, UC5, UC6, UC7)
DFS (Depth-First Search)	§3 (UC4), §6, Annexe A
Diagramme SVN («SVNDiagram»)	§2, §3 (UC2, UC5, UC6, UC7), §4
importanceScore	§3 (UC4, UC5), §4
IBM Rhapsody	§1, §2, §5, §7
INCOSE 2018	§3 (UC4), §7, §8, Annexe B
IRPActor	§4, §5
IRPApplication	§2, §5
IRPCollection	§1, §4
IRPFlow	§3 (UC2, UC3), §4, §5

IRPProject	§2, §5
Java 8 / JDK 1.8	§2, §5, §7
JColorChooser	§3 (UC6), §5
JOptionPane	§3 (UC3), §5
Maven	§2, §5, §7
Nœud système («system»)	§2, §3 (UC4), §4
ProfileService	§2, §3 (UC1, UC8)
RhapsodyWrapper	§2, §7
RPUUserPlugin	§1, §2
supplyImportance	§3 (UC3, UC4), §4, Annexe B
SupplyImportance	§3 (UC3, UC4), §4, Annexe B
SVNCalculateCommand	§2, §3 (UC4)
SVNCleanCommand	§2, §3 (UC8)
SVNConfigureCommand	§2, §3 (UC1)
SVNConstants	§2, §7
SVNPlugin	§1, §2, §5

SVNProfile	§2, §3 (UC1, UC8), §4
Stakeholder («stakeholder»)	§2, §3 (UC4, UC5), §4
Stéréotype	§1, §2, §3, §4
SysML	§1, §2
Tag	§1, §3, §4
Value loop	§1, §3 (UC4), §6, Annexe A
Windows	§2, §5

Index des fonctions et classes

Classe / Méthode	Rôle	Section(s)
SVNPlugin	Point d'entrée du plugin, route les commandes	§2, §5
SVNConfigureCommand.run()	Lance la configuration du profil SVN	§3 (UC1)
SVNCreateArcCommand.run()	Crée un arc «valuearc» entre deux éléments sélectionnés	§3 (UC2)
SVNEditArcCommand.run()	Édite les pondérations d'un arc sélectionné	§3 (UC3)
SVNCalculateCommand.run()	Lance le calcul d'importance SVN	§3 (UC4)
SVNColorizeStakeholdersCommand.run()	Colorise les stakeholders par rang	§3 (UC5)
SVNArcColorCommand.run()	Applique une couleur à un arc	§3 (UC6)
SVNLabelArcCommand.run()	Met à jour les labels d'arcs	§3 (UC7)
SVNCleanCommand.run()	Nettoie le modèle SVN	§3 (UC8)
ProfileService.configureProfile()	Crée/recrée le profil SVNProfile	§3 (UC1)
ProfileService.cleanDefaultPackage()	Supprime les éléments SVN du package Default	§3 (UC8)
CalculationService.calculateImportance()	Orchestre le calcul d'importance	§3 (UC4)

<code>CalculationService.calculateByValueLoops()</code>	Calcul principal par DFS	§3 (UC4), Annexe A
<code>CalculationService.calculateByArcSum()</code>	Calcul simplifié (fallback)	§3 (UC4)
<code>DiagramService.createArcBetweenSelected()</code>	Crée un arc entre éléments graphiques	§3 (UC2)
<code>DiagramService.colorizeStakeholdersByRank()</code>	Colorise les nœuds par tier	§3 (UC5)
<code>DiagramService.setArcColor()</code>	Change la couleur d'un arc	§3 (UC6)
<code>DiagramService.updateArcLabels()</code>	Met à jour les labels d'arcs	§3 (UC7)
<code>RhapsodyWrapper.hasStereotype()</code>	Vérifie si un élément porte un stéréotype donné	§2, §3
<code>RhapsodyWrapper.getOrCreateStereotype()</code>	Crée ou récupère un stéréotype	§3 (UC1)
<code>RhapsodyWrapper.getOrCreateEnumType()</code>	Crée ou récupère un type énuméré	§3 (UC1)

10. Annexes

Annexe A — Méthode SVN et équations de Cameron (2007)

Présentation de la méthode

La méthode **Stakeholder Value Network (SVN)** est une approche d'ingénierie système permettant d'analyser les relations de valeur entre les parties prenantes d'un système. Elle représente ces relations sous forme d'un réseau orienté (graphe) où :

- Les **nœuds** sont des parties prenantes («stakeholder») et le système central («system»)
- Les **arcs** («valuearc») représentent des flux de valeur orientés entre deux entités

Scores des arcs

Chaque arc est caractérisé par deux attributs :

- **benefitRanking**: importance du bénéfice apporté par le flux du point de vue du destinataire (MIGHT_BE, SHOULD_BE, MUST_BE)
- **supplyImportance**: capacité du fournisseur à assurer ce flux (LOW, MEDIUM, HIGH)

Le score d'un arc est obtenu par la matrice INCOSE 2018 (voir Annexe B).

Notion de Value Loop

Un **value loop** est un cycle dans le graphe SVN qui commence et se termine sur le nœud «system». Il représente un circuit de création de valeur impliquant le système et un sous-ensemble de parties prenantes.

Exemple : `system → stakeholder_A → stakeholder_B → system`

Équation 1 — Score d'un value loop

Le score d'un value loop L est le **produit des scores de ses arcs**:

$$\text{score}(L) = \prod \text{score}(\text{arc}_i) \quad \text{pour tout arc}_i \text{ appartenant à } L$$

Cette formule traduit le fait qu'un loop n'est efficace que si tous ses arcs sont de bonne qualité.

Équation 2 — Importance d'un stakeholder

L'importance relative d'un stakeholder S est calculée comme le **rapport entre la somme des scores des loops contenant S et la somme des scores de tous les loops**:

$$\text{importance}(S) = \frac{\sum \text{score}(L_i) \text{ [pour } L_i \text{ contenant } S]}{\sum \text{score}(L_j) \text{ [tous } L_j]}$$

Le score obtenu est normalisé entre 0 et 1. Un stakeholder impliqué dans de nombreux loops de haute valeur obtient un score proche de 1.

Fallback — Calcul simplifié par somme des arcs

En l'absence de nœud «system», aucune valeur loop ne peut être détectée. Le plugin bascule sur un calcul simplifié :

$$\begin{aligned} \text{score_brut}(S) &= \sum \text{score}(\text{arc}_i) \text{ [pour tout arc}_i \text{ adjacent à } S] \\ \text{importance}(S) &= \text{score_brut}(S) / \sum \text{score_brut}(S_j) \text{ [tous les stakeholders]} \end{aligned}$$

Ce calcul est moins précis que la méthode principale mais reste fonctionnel pour des modèles sans système central défini.

Annexe B — Matrice de score des arcs (INCOSE 2018, Figure 3)

La matrice ci-dessous définit le score numérique associé à chaque combinaison (benefitRanking, supplyImportance). Elle est implémentée dans CalculationService.getArcScore().

supplyImportance ↓ \ benefitRanking →	MIGHT_BE	SHOULD_BE	MUST_BE
HIGH	0.30	0.50	0.95
MEDIUM	0.20	0.40	0.80
LOW	0.10	0.20	0.40

Lecture de la matrice :

- Un arc dont le bénéfice est critique (MUST_BE) et dont la capacité de fourniture est élevée (HIGH) obtient le score maximal de **0.95**
- Un arc dont le bénéfice est optionnel (MIGHT_BE) avec une faible capacité de fourniture (LOW) obtient le score minimal de **0.10**
- La valeur par défaut (tags absents ou invalides) est **0.20** (équivalent à MIGHT_BE / MEDIUM)

Annexe C — Structure du projet Maven

```

RhapsodySVN/
├── pom.xml                    # Configuration Maven (Java
8, dépendance rhapsody.jar)
├── lib/
│   ├── rhapsody.jar          # API IBM Rhapsody 9.0
(dépendance locale)
│   └── 64Bit/
│       └── rhapsody.dll       # Bibliothèque native Windows
└── src/main/java/fr/utbm/RhapsodySVN/
    ├── SVNPlugin.java        # Point d'entrée plugin
(RPUserPlugin)
    ├── SVNConfigureCommand.java # UC1 – Configurer le profil
    ├── SVNCreateArcCommand.java # UC2 – Créer un arc
    └── SVNEditArcCommand.java   # UC3 – Éditer les
pondérations

```

```

    └─ SVNCalculateCommand.java      # UC4 – Calculer l'importance
    └─ SVNColorizeStakeholdersCommand.java # UC5 – Coloriser les
stakeholders
    └─ SVNArcColorCommand.java      # UC6 – Coloriser un arc
    └─ SVNLabelArcCommand.java      # UC7 – Mettre à jour les
labels
    └─ SVNCleanCommand.java         # UC8 – Nettoyer le modèle
    └─ Main.java                   # Point d'entrée ligne de
commande
    └─ constants/
        └─ SVNConstants.java        # Centralisation des
constantes
    └─ rhapsody/
        └─ RhapsodyWrapper.java     # Wrapper bas niveau API
Rhapsody
    └─ service/
        └─ ProfileService.java      # Gestion du profil
SVNProfile
    └─ CalculationService.java      # Algorithme de calcul SVN
    └─ DiagramService.java         # Opérations graphiques sur
les diagrammes

```

Annexe D — Procédure d'installation et de premier lancement

1. **Pré-requis:** IBM Rhapsody 9.0 installé sur Windows, JDK 1.8, Maven 3.x
2. **Compilation:** `mvn clean package` — produit `target/RhapsodySVN-1.0-SNAPSHOT.jar`
3. **Déploiement:** placer le JAR dans le répertoire plugins de Rhapsody et configurer le fichier de configuration Rhapsody pour déclarer `fr.utbm.RhapsodySVN.SVNPlugin` comme plugin actif
4. **Premier lancement:**
 - Ouvrir IBM Rhapsody avec un projet actif
 - Exécuter `SVN Configure` depuis le menu pour initialiser le profil

- Créer les éléments «stakeholder», «system» et «valuearc» manuellement dans le diagramme
- Renseigner les pondérations via SVN Edit Arc (UC3) pour chaque arc
- Lancer SVN Calculate (UC4) pour calculer les scores
- Visualiser les résultats via SVN Colorize Stakeholders (UC5) et SVN Update Arc Labels (UC7)